Training and generalization in overparameterized neural networks

Stage 1 review

Shreyas Kalvankar

January 29, 2026

TU Delft

Agenda

Agenda

1. Introduction & Problem setup
2. Related work
3. Research Questions & Objectives
4. Preliminary observations
5. Planned work & Discussions

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Data: n observations $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Prediction function: $f(x) \in \mathbb{R}$
- Task: Find the function f^*
- Objective: that minimizes the empirical risk:

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Data: n observations $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Prediction function: $f(x) \in \mathbb{R}$
- Task: Find the function f^*
- Objective: that minimizes the empirical risk:

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Data: n observations $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Prediction function: $f(x) \in \mathbb{R}$
- Task: Find the function f^*
- Objective: that minimizes the empirical risk:

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Data: n observations $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Prediction function: $f(x) \in \mathbb{R}$
- Task: Find the function f^*
- Objective: that minimizes the empirical risk: $\mathcal{L}(f) = \sum_{i=1}^{n} d(f(x_i), y_i) = \sum_{i=1}^{n} (f(x_i) - y_i)^2$

Introduction & Problem setup

Problem setup: Empirical Risk Minimization

- Data: n observations $\{(x_i, y_i)\}_{i=1}^n$, with inputs $x_i \in \mathbb{R}^d$ and targets $y_i \in \mathbb{R}$ (stacked as $X \in \mathbb{R}^{d \times n}, y \in \mathbb{R}^n$).
- Prediction function: $f(x) \in \mathbb{R}$
- Task: Find the function f^*
- Objective: that minimizes the empirical risk: $\mathcal{L}(f) = d(f_\theta(X), Y) = \|(f(X) - y)\|^2$

Introduction & Problem setup

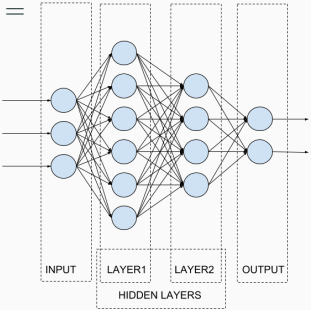
In order to minimize the empirical risk we consider a family of parameterized functions $f_\theta : \mathbb{R}^d \to R$:

- Linear Networks: $f_\theta(x) = x^\top \theta$
- Neural Networks: $f_\theta(x) =$

Introduction & Problem setup

In order to minimize the empirical risk we consider a family of parameterized functions $f_\theta : \mathbb{R}^d \to R$:

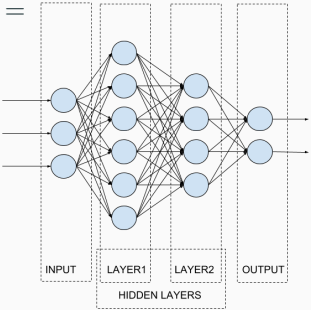
- Linear Networks: $f_\theta(x) = x^\top \theta$
- Neural Networks: $f_\theta(x) =$



Introduction & Problem setup

In order to minimize the empirical risk we consider a family of parameterized functions $f_\theta : \mathbb{R}^d \to R$:

- Linear Networks: $f_\theta(x) = x^\top \theta$
- Neural Networks: $f_\theta(x) =$



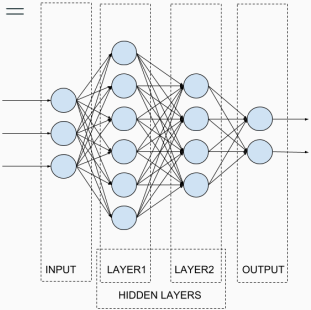
Expressivity power

Introduction & Problem setup

In order to minimize the empirical risk we consider a family of parameterized functions $f_\theta : \mathbb{R}^d \to R$:

- Linear Networks: $f_\theta(x) = x^\top \theta$
- Neural Networks: $f_\theta(x) =$

Simplicity of analysis



Expressivity power

Why Analyze Training Dynamics?

- Neural networks are trained using gradient-based methods on high-dimensional, non-convex objectives.
- Understanding **training dynamics** is key to explaining convergence behavior and generalization.
- However, for nonlinear networks like neural networks, these dynamics are difficult to characterize analytically.

Why Analyze Training Dynamics?

- Neural networks are trained using gradient-based methods on high-dimensional, non-convex objectives.
- Understanding **training dynamics** is key to explaining convergence behavior and generalization.
- However, for nonlinear networks like neural networks, these dynamics are difficult to characterize analytically.

Why Analyze Training Dynamics?

- Neural networks are trained using gradient-based methods on high-dimensional, non-convex objectives.
- Understanding **training dynamics** is key to explaining convergence behavior and generalization.
- However, for nonlinear networks like neural networks, these dynamics are difficult to characterize analytically.

Why Are Nonlinear Dynamics Hard?

- For linear models, gradient descent induces a linear dynamical system where convergence can be explained by the spectrum of the covariance matrix (operator) $XX^\top$.

$$f_w(x) = x^\top w \qquad \dot{w}_t = -\nabla_w \mathcal{L}(w_t) = Xy - XX^\top w_t$$

- Introducing depth to linear models already leads to coupled, nonlinear dynamics with no general closed-form solution.

$$f_{v,W}(x) = (Wx)^\top v \qquad \begin{aligned} \dot{v}_t &= -\nabla_v \mathcal{L}(v_t, W_t) = W_t X \left(y - X^\top W_t^\top v_t\right) \\ \dot{W}_t &= -\nabla_W \mathcal{L}(v_t, W_t) = v_t \left(y - X^\top W_t^\top v_t\right)^\top X^\top. \end{aligned}$$

- Adding nonlinear activations mixes matrix products with elementwise nonlinearities.

$$f_{v,W}(x) = \phi(Wx)^\top v \qquad \begin{aligned} \dot{v}_t &= \phi(W_t X)\left(y - \phi(W_t X)^\top v_t\right), \\ \dot{W}_t &= \left(\phi'(W_t X) \odot v_t (y - \phi(W_t X)^\top v_t)^\top\right) X^\top. \end{aligned}$$

- We don't know of any mathematical tools to analyse such systems.

Why Are Nonlinear Dynamics Hard?

- For linear models, gradient descent induces a linear dynamical system where convergence can be explained by the spectrum of the covariance matrix (operator) $XX^\top$.

$$f_w(x) = x^\top w \qquad \dot{w}_t = -\nabla_w \mathcal{L}(w_t) = Xy - XX^\top w_t$$

- Introducing depth to linear models already leads to coupled, nonlinear dynamics with no general closed-form solution.

$$f_{v,W}(x) = (Wx)^\top v$$

$$\dot{v}_t = -\nabla_v \mathcal{L}(v_t, W_t) = W_t X \big(y - X^\top W_t^\top v_t\big)$$
$$\dot{W}_t = -\nabla_W \mathcal{L}(v_t, W_t) = v_t \big(y - X^\top W_t^\top v_t\big)^\top X^\top.$$

- Adding nonlinear activations mixes matrix products with elementwise nonlinearities.

$$f_{v,W}(x) = \phi(Wx)^\top v$$

$$\dot{v}_t = \phi(W_t X)\big(y - \phi(W_t X)^\top v_t\big),$$
$$\dot{W}_t = \big(\phi'(W_t X) \odot v_t (y - \phi(W_t X)^\top v_t)^\top\big) X^\top.$$

- We don't know of any mathematical tools to analyse such systems.

Why Are Nonlinear Dynamics Hard?

- For linear models, gradient descent induces a linear dynamical system where convergence can be explained by the spectrum of the covariance matrix (operator) $XX^\top$.

$$f_w(x) = x^\top w \qquad \dot{w}_t = -\nabla_w \mathcal{L}(w_t) = Xy - XX^\top w_t$$

- Introducing depth to linear models already leads to coupled, nonlinear dynamics with no general closed-form solution.

$$f_{v,W}(x) = (Wx)^\top v \qquad \begin{aligned} \dot{v}_t &= -\nabla_v \mathcal{L}(v_t, W_t) = W_t X \big(y - X^\top W_t^\top v_t\big) \\ \dot{W}_t &= -\nabla_W \mathcal{L}(v_t, W_t) = v_t \big(y - X^\top W_t^\top v_t\big)^\top X^\top. \end{aligned}$$

- Adding nonlinear activations mixes matrix products with elementwise nonlinearities.

$$f_{v,W}(x) = \phi(Wx)^\top v \qquad \begin{aligned} \dot{v}_t &= \phi(W_t X)\big(y - \phi(W_t X)^\top v_t\big), \\ \dot{W}_t &= \big(\phi'(W_t X) \odot v_t (y - \phi(W_t X)^\top v_t)^\top\big) X^\top. \end{aligned}$$

- We don't know of any mathematical tools to analyse such systems.

Why Are Nonlinear Dynamics Hard?

- For linear models, gradient descent induces a linear dynamical system where convergence can be explained by the spectrum of the covariance matrix (operator) $XX^\top$.

$$f_w(x) = x^\top w \qquad\qquad \dot{w}_t = -\nabla_w \mathcal{L}(w_t) = Xy - XX^\top w_t$$

- Introducing depth to linear models already leads to coupled, nonlinear dynamics with no general closed-form solution.

$$f_{v,W}(x) = (Wx)^\top v \qquad \begin{aligned} \dot{v}_t &= -\nabla_v \mathcal{L}(v_t, W_t) = W_t X \big(y - X^\top W_t^\top v_t\big) \\ \dot{W}_t &= -\nabla_W \mathcal{L}(v_t, W_t) = v_t \big(y - X^\top W_t^\top v_t\big)^\top X^\top. \end{aligned}$$

- Adding nonlinear activations mixes matrix products with elementwise nonlinearities.

$$f_{v,W}(x) = \phi(Wx)^\top v \qquad \begin{aligned} \dot{v}_t &= \phi(W_t X)\big(y - \phi(W_t X)^\top v_t\big), \\ \dot{W}_t &= \big(\phi'(W_t X) \odot v_t (y - \phi(W_t X)^\top v_t)^\top\big) X^\top. \end{aligned}$$

- We don't know of any mathematical tools to analyse such systems.

Related work

Solution: Take limits!

- In the infinite-width limit, neural network training can be linearized around initialization.
- The resulting dynamics are linear and governed by the **Neural Tangent Kernel (NTK)**.
- Training becomes equivalent to kernel gradient descent with a fixed kernel.
- Convergence behavior is governed by the spectrum of the NTK operator.

Image Sources: DOI:10.1137/S0895479895290954; Katie M., AQA A Level Physics Notes

Solution: Take limits!

- In the infinite-width limit, neural network training can be linearized around initialization.
- The resulting dynamics are linear and governed by the **Neural Tangent Kernel (NTK)**.
- Training becomes equivalent to kernel gradient descent with a fixed kernel.
- Convergence behavior is governed by the spectrum of the NTK operator.

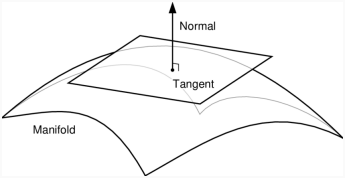


Image Sources: DOI:10.1137/S0895479895290954; Katie M., AQA A Level Physics Notes

Solution: Take limits!

- In the infinite-width limit, neural network training can be linearized around initialization.
- The resulting dynamics are linear and governed by the **Neural Tangent Kernel (NTK)**.
- Training becomes equivalent to kernel gradient descent with a fixed kernel.
- Convergence behavior is governed by the spectrum of the NTK operator.

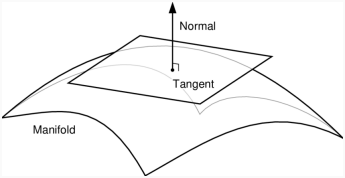
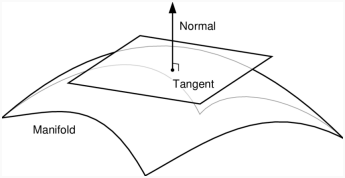


Image Sources: DOI:10.1137/S0895479895290954; Katie M., AQA A Level Physics Notes

Solution: Take limits!

- In the infinite-width limit, neural network training can be linearized around initialization.
- The resulting dynamics are linear and governed by the **Neural Tangent Kernel (NTK)**.
- Training becomes equivalent to kernel gradient descent with a fixed kernel.
- Convergence behavior is governed by the spectrum of the NTK operator.



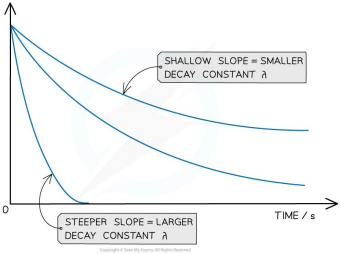


Image Sources: DOI:10.1137/S0895479895290954; Katie M., AQA A Level Physics Notes

Spectral Bias in the NTK Regime

- A well-known empirical phenomenon in neural network training is **spectral bias**: low-frequency components are learned faster than high-frequency ones.
- In the NTK limit, this phenomenon admits a precise theoretical explanation.
- For ReLU networks trained on a continuum domain (e.g. $[0, 2\pi]$), the NTK operator is diagonal in the Fourier basis.
- Eigenvalues are ordered by frequency: lower frequencies correspond to larger eigenvalues.

Image Sources: Overview Spectral Bias in Deep Learning, vol 7; Katie M., AQA A Level Physics Notes

Spectral Bias in the NTK Regime

- A well-known empirical phenomenon in neural network training is **spectral bias**: low-frequency components are learned faster than high-frequency ones.
- In the NTK limit, this phenomenon admits a precise theoretical explanation.
- For ReLU networks trained on a continuum domain (e.g. $[0, 2\pi]$), the NTK operator is diagonal in the Fourier basis.
- Eigenvalues are ordered by frequency: lower frequencies correspond to larger eigenvalues.

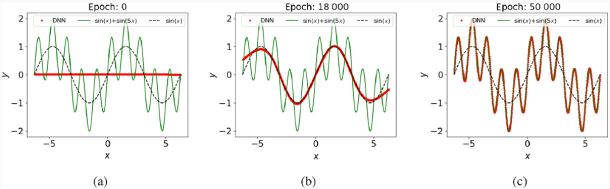


Image Sources: Overview Spectral Bias in Deep Learning, vol 7; Katie M., AQA A Level Physics Notes

Spectral Bias in the NTK Regime

- A well-known empirical phenomenon in neural network training is **spectral bias**: low-frequency components are learned faster than high-frequency ones.
- In the NTK limit, this phenomenon admits a precise theoretical explanation.
- For ReLU networks trained on a continuum domain (e.g. $[0, 2\pi]$), the NTK operator is diagonal in the Fourier basis.
- Eigenvalues are ordered by frequency: lower frequencies correspond to larger eigenvalues.

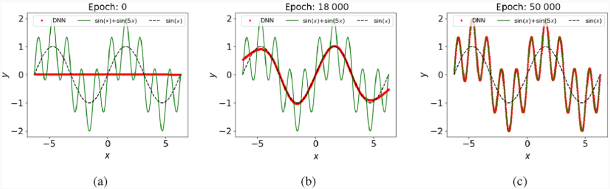
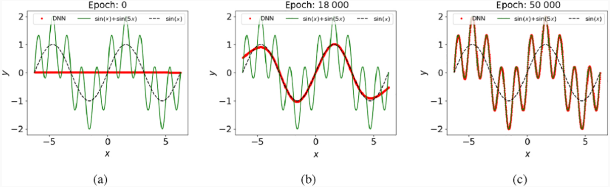


Image Sources: Overview Spectral Bias in Deep Learning, vol 7; Katie M., AQA A Level Physics Notes

Spectral Bias in the NTK Regime

- A well-known empirical phenomenon in neural network training is **spectral bias**: low-frequency components are learned faster than high-frequency ones.
- In the NTK limit, this phenomenon admits a precise theoretical explanation.
- For ReLU networks trained on a continuum domain (e.g. $[0, 2\pi]$), the NTK operator is diagonal in the Fourier basis.
- Eigenvalues are ordered by frequency: lower frequencies correspond to larger eigenvalues.



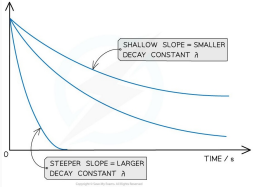


Image Sources: Overview Spectral Bias in Deep Learning, vol 7; Katie M., AQA A Level Physics Notes

The Gap: Beyond the Ideal NTK

- Spectral bias is observed empirically in **finite-width** neural networks.
- Existing theoretical results characterize spectral bias in the **infinite-width, population NTK limit**, where training dynamics are governed by a deterministic integral operator.
- While finite-width and finite-sample networks are known to track NTK dynamics to some extent, it remains unclear how **frequency-wise learning dynamics** are quantitatively affected by finite width and finite sample size.

The Gap: Beyond the Ideal NTK

- Spectral bias is observed empirically in **finite-width** neural networks.
- Existing theoretical results characterize spectral bias in the **infinite-width, population NTK limit**, where training dynamics are governed by a deterministic integral operator.
- While finite-width and finite-sample networks are known to track NTK dynamics to some extent, it remains unclear how **frequency-wise learning dynamics** are quantitatively affected by finite width and finite sample size.

The Gap: Beyond the Ideal NTK

- Spectral bias is observed empirically in **finite-width** neural networks.
- Existing theoretical results characterize spectral bias in the **infinite-width, population NTK limit**, where training dynamics are governed by a deterministic integral operator.
- While finite-width and finite-sample networks are known to track NTK dynamics to some extent, it remains unclear how **frequency-wise learning dynamics** are quantitatively affected by finite width and finite sample size.

Research objective

Common Setting and Assumptions

- **Data:** inputs $x(\theta) = (\cos\theta, \sin\theta)$ on S^1, sampled uniformly unless stated otherwise.
- **Target functions:** finite Fourier mixtures of the form

$$f^\star(\theta) = \sum_{k \in \mathcal{K}} a_k \sin(k\theta),$$

  with a finite frequency set $\mathcal{K}$ and fixed coefficients.
- **Model:** two-layer ReLU network trained by gradient descent.
- **Training:** fixed learning rate and fixed number of training steps.

Main Research Question

Main Question. To what extent do the frequency-wise training dynamics predicted by the ideal infinite-width, population NTK persist when either the number of training samples or the network width is finite?

More precisely:

- how are the decay rates of individual Fourier components of the training error affected by finite sample size?
- how are these frequency-wise dynamics perturbed by finite network width when training population is effectively infinite?

Main Research Question

Main Question. To what extent do the frequency-wise training dynamics predicted by the ideal infinite-width, population NTK persist when either the number of training samples or the network width is finite?

More precisely:

- how are the decay rates of individual Fourier components of the training error affected by finite sample size?
- how are these frequency-wise dynamics perturbed by finite network width when training population is effectively infinite?

Research Question Q1: Finite-Sample Frequency Dynamics

Setting. Consider a two-layer ReLU network in the infinite-width NTK regime, with inputs on S^1 and uniform sampling. The population NTK defines a convolution operator T diagonalized by Fourier modes.

Question. What happens to the frequency-wise error decay when the population NTK operator T is replaced by its finite-sample discretization T_n?

Approach. We discretize the NTK integral operator via its Mercer decomposition and analyze the resulting finite-dimensional linear system governing the evolution of Fourier coefficients of the residual.

Focus. We examine how finite sampling affects:

- the rate of decay of error projected onto individual Fourier modes,
- deviations from ideal exponential decay,
- and interactions between distinct frequency components induced by discretization.

Research Question Q1: Finite-Sample Frequency Dynamics

Setting. Consider a two-layer ReLU network in the infinite-width NTK regime, with inputs on S^1 and uniform sampling. The population NTK defines a convolution operator T diagonalized by Fourier modes.

Question. What happens to the frequency-wise error decay when the population NTK operator T is replaced by its finite-sample discretization T_n?

Approach. We discretize the NTK integral operator via its Mercer decomposition and analyze the resulting finite-dimensional linear system governing the evolution of Fourier coefficients of the residual.

Focus. We examine how finite sampling affects:

- the rate of decay of error projected onto individual Fourier modes,
- deviations from ideal exponential decay,
- and interactions between distinct frequency components induced by discretization.

Research Question Q2: Finite-Width Frequency Dynamics

Setting. Consider a two-layer ReLU network on S^1 in a regime where the NTK is approximately constant. The infinite-width NTK defines a convolution operator T diagonalized by Fourier modes.

Question. What happens to the frequency-wise error decay when the population NTK operator T is replaced by the random finite-width NTK operator $T^{(m)}$ induced by m neurons?

Approach. Treat $T^{(m)}$ as a random perturbation of T, project the residual dynamics onto Fourier modes, and analyze the resulting coupled linear system for Fourier coefficients.

Focus. We examine how finite width affects:

- the decay of error projected onto individual Fourier modes,
- deviations from ideal exponential decay rates,
- and mode interactions induced by finite-width NTK.

Research Question Q2: Finite-Width Frequency Dynamics

Setting. Consider a two-layer ReLU network on S^1 in a regime where the NTK is approximately constant. The infinite-width NTK defines a convolution operator T diagonalized by Fourier modes.

Question. What happens to the frequency-wise error decay when the population NTK operator T is replaced by the random finite-width NTK operator $T^{(m)}$ induced by m neurons?

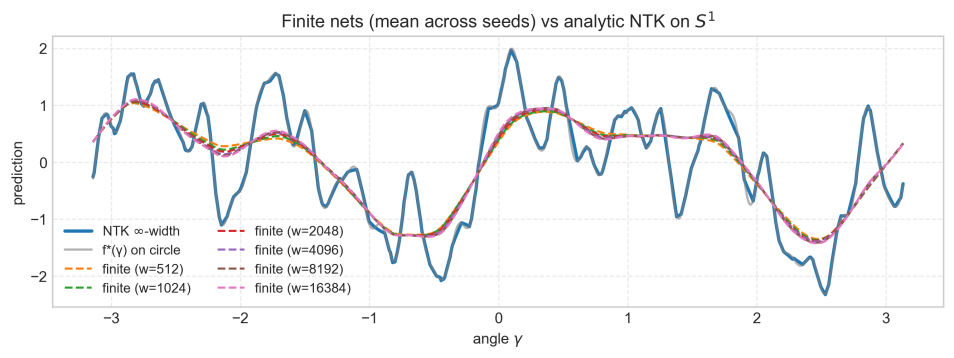
Approach. Treat $T^{(m)}$ as a random perturbation of T, project the residual dynamics onto Fourier modes, and analyze the resulting coupled linear system for Fourier coefficients.

Focus. We examine how finite width affects:

- the decay of error projected onto individual Fourier modes,
- deviations from ideal exponential decay rates,
- and mode interactions induced by finite-width NTK.

Preliminary observations

Spectral Bias on Fourier Mixtures (EXP001)

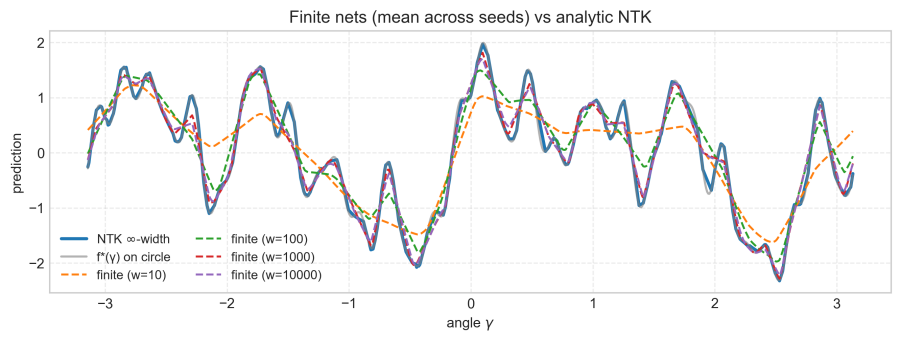


Learned functions (trained with learning rate = 1e-3, 30k training steps).

Observation. Learned functions are dominated by low frequencies under a fixed training horizon, with only weak dependence on width.

Interpretation. High-frequency components converge slowly due to small NTK eigenvalues, motivating frequency-resolved analysis.

Delayed Learning of High Frequencies (EXP001)

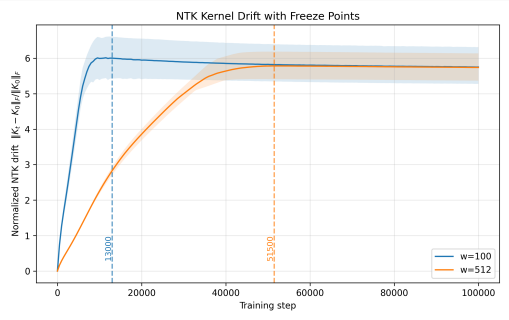


Learned functions (trained with learning rate $= 1$, 100k training steps).

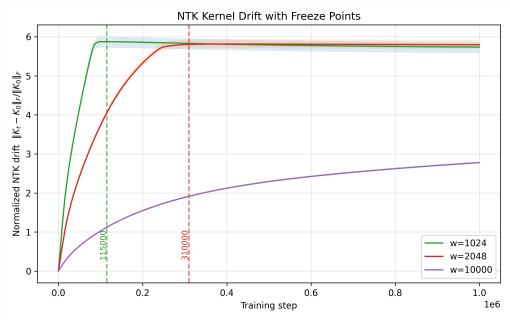
Observation. With larger effective training ηt, higher-frequency structure emerges, even at small widths.

Interpretation. Spectral bias reflects slow convergence of small-eigenvalue modes, not merely representational limits.

Empirical NTK Drift (Low and Mid Widths) (EXP002)

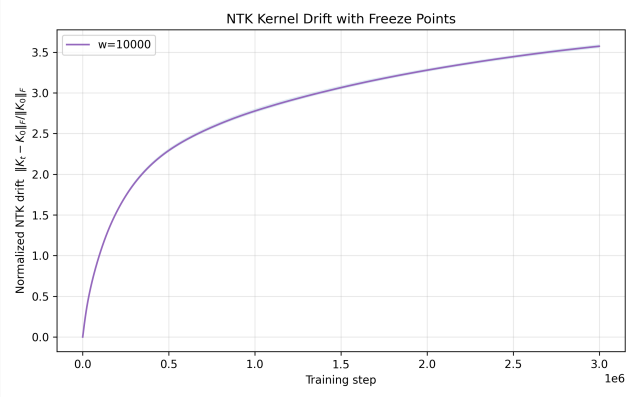


(a) Low widths.



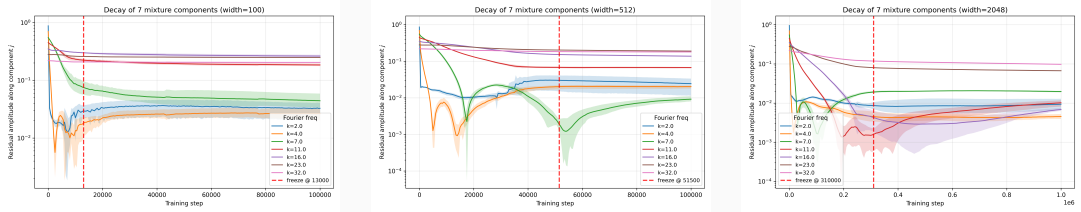
(b) Mid widths.

Empirical NTK Drift (High Width) (EXP002)



(c) High width.

Mode-wise Residual Decay (EXP003)



(a) m=100 (b) m=512 (c) m=2048

Observation. Low-frequency Fourier components decay rapidly, while high-frequency components persist for much longer across widths.

Interpretation. This provides a direct, frequency-resolved signal of frequency bias and motivates Q1 (finite samples) and Q2 (finite width).

Planned Timeline and Diagnostics

- **February: Analytic groundwork (Q1)**
  - Analytically derive the exact operator NTK operator and the convolution operator T for 2-layer ReLU network on S^1.
  - Formulate the finite-sample discretization T_n and the induced coupled Fourier-mode dynamics.
  - Identify diagnostic quantities: frequency component-wise residual decay and off-diagonal coupling between frequencies.
- **March: Finite-sample experiments (Q1)**
  - Simulate discretized kernel dynamics for varying sample sizes n.
  - Measure effective decay rates for each frequency component.
  - Quantify deviations from ideal exponential decay and onset of frequency coupling.

Planned Timeline and Diagnostics

- **April: Finite-width experiments (Q2)**
  - Sweep network width m with effectively infinite samples based on results obtained from Q1, with fixed learning rate and training time.
  - Track residual coefficients of each frequency component across seeds.
  - Measure coupling of the frequency-components.
- **April final weeks: Integration and write-up**
  - Compare finite-sample and finite-width effects on frequency dynamics.
  - Consolidate results, refine figures, and finalize thesis chapters.

Thank you

Questions?